[UNIVERSITY / BOARD NAME]

A

PROJECT REPORT ON

"PARKARO"

Smart Parking Finder, Reservation and Payment System

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE OF

BACHELOR OF ENGINEERING IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBMITTED BY

Sr. No.	First Name	Middle Name	Surname	Enrollment / Roll No.
1.	Anuj	Bajarang	Sutar	

UNDER THE GUIDANCE OF

[Guide name]

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

[COLLEGE NAME, PLACE, DISTRICT, STATE – PIN]

ACADEMIC YEAR [20__–20__]

This undertaken project work is the record of the students' own work carried out by them during the academic year 2026-2027.

[COLLEGE NAME]

CERTIFICATE

This is to certify that **Anuj Bajarang Sutar** ([Roll No.]),have successfully completed and submitted the project report titled **"Parkaro – Smart Parking Finder, Reservation and Payment System"** in partial fulfilment for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Data Science of [University name].

[Guide name] (Guide)	[HOD name] (Head of Department)
[Project Coordinator] (Project Coordinator)	(External Examiner)
[Principal / Director] (Principal)	

Institute and Department: Vision, Mission, PEO and PSO

[Paste the vision, mission, PEOs and PSOs of your own institute and department here. The old report listed those of the diploma institute, which no longer apply to this report.]

ACKNOWLEDGEMENT

We express our gratitude to everyone who supported us throughout the course of this project. We are thankful for the guidance and friendly advice that we received during the project work, and we are sincerely grateful for the honest and useful views they shared on many issues related to the project.

We express our warm thanks to our Principal [Principal name] and our Head of Department [HOD name] for their guidance and direction towards our final year project. We are very thankful to our Project Coordinator [Coordinator name] for the valuable co-operation and guidance during this project. We would also like to thank our Project Guide [Guide name] and all the people who provided us with the facilities required for our project.

This project grew out of our earlier diploma project “Parkaro”. We also thank our former diploma guide and teachers, whose feedback on that first version helped us decide what a better version should do. The whole team of the Department of Artificial Intelligence and Data Science helped us throughout the project and also helped us to clear our obstacles. It is a great honour to be students of [College name] and to get several opportunities to learn something new every day.

ABSTRACT

Finding a safe place to park is a daily problem in growing towns. Drivers circle streets looking for a space, owners of private land have no easy way to rent it out, and most small-town parking is still paid in cash with a paper token. **Parkaro** is a web application that addresses this for the cities of Kolhapur, Ajara and Gadhinglaj in Maharashtra. It lets a driver find parking near their location, see a map, get directions, and reserve a private parking space for a chosen time window; and it lets a parking owner register their own place, decide how customers may pay, and see verified earnings.

This report describes the second, upgraded version of Parkaro. The first version (a diploma project) displayed parking locations on a map and stored simple bookings. The upgraded version adds a complete reservation and payment workflow: the server calculates the price, checks capacity and holds the space for 20 minutes; payment can be made online through Razorpay (UPI, cards and net banking) or at the place in cash or by UPI paid directly to the owner; and a reservation becomes "confirmed" only after the payment has been verified on the server — by a signed callback and a lookup with the payment gateway, a signed webhook, or the owner's recorded confirmation. After confirmation the customer receives a ticket by e-mail and, if they choose, by WhatsApp and SMS. Each message is sent once per channel, delivery status is recorded, failures never change the booking or the payment, and the owner can retry.

The system is built with HTML, CSS, JavaScript, Leaflet.js with OpenStreetMap, PHP 8 and MySQL/MariaDB, and runs on a local XAMPP server. Security is treated as a design goal: prepared statements, hashed passwords, CSRF tokens, signed ticket links, owner-scoped access to data, and idempotent payment handling. The project is checked by an automated test suite of 938 checks (all passing at the time of writing) that uses a SQLite copy of the schema and fake payment and message providers. Live delivery through real Razorpay, SMTP, Twilio and WhatsApp accounts has not been tested and needs the credentials described in the setup guide.

UNDERTAKING

We hereby confirm that the work presented in the final year project report entitled **"Parkaro – Smart Parking Finder, Reservation and Payment System"**, in fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Data Science** and submitted to the Department of Artificial Intelligence and Data Science of [College name], is an authentic record of our own work carried out during the period from [month year] to [month year] under the supervision of [Guide name].

The project builds on our earlier diploma project "Parkaro" (academic year 2024-25); the additions made in this version are listed in Section 1.5.

Signature of Students

Mr. ANUJ BAJARANG SUTAR .....

INDEX

ACKNOWLEDGEMENT 4
ABSTRACT 5
UNDERTAKING 6
CHAPTER – 1: INTRODUCTION & BACKGROUND 9
1.1 Project Preface 9
1.2 Detailed Problem Definition 9
1.3 Existing System and Its Limitations 10
1.4 Objectives 10
1.5 What Is New Compared With the Diploma Version 11
CHAPTER – 2: ANALYSIS 12
2.1 System Analysis 12
2.2 Requirement Specification 12
2.3 Purpose 13
2.4 Scope 13
2.5 Feasibility 14
CHAPTER – 3: DESIGN 15
3.1 Block Diagram 15
3.2 Flowchart 15
3.3 Use Case Diagram 17
3.4 Reservation and Payment States 17
3.5 Ticket Message Flow 18
CHAPTER – 4: SYSTEM MODELING 20
4.1 Data Flow Diagram 20
4.2 ER Diagram 22
4.3 Data Dictionary (main tables) 22
CHAPTER – 5: RELATIVE THEORY 24
5.1 Technologies Used 24
CHAPTER – 6: IMPLEMENTATION 26
6.1 Project Structure 26
6.2 Reservation Module 26
6.3 Single Confirmation Point 26
6.4 Online Payment Module 27
6.5 Cash and UPI Confirmation by the Owner 27
6.6 Notification Module 28
6.7 Find Parking, Map and City Handling 28
6.8 Database Migrations 28
6.9 Configuration 28
CHAPTER – 7: TESTING 29
7.1 Testing Approach 29
7.2 Test Results 29
7.3 Important Scenarios Tested 29
7.4 What Was Not Tested 30
CHAPTER – 8: OUTPUT 31

CHAPTER – 9: FEATURES.....39
CHAPTER – 10: LIMITATIONS.....40
CHAPTER – 11: FUTURE SCOPE.....41
CHAPTER – 12: CONCLUSION.....42
CHAPTER – 13: REFERENCES.....43
APPENDIX A: SETUP AND MIGRATION ON XAMPP.....45
APPENDIX B: CONFIGURATION (.env) – NO REAL SECRETS HERE.....46
APPENDIX C: MANUAL TESTING CHECKLIST (REAL SERVICES).....47

CHAPTER – 1: INTRODUCTION & BACKGROUND

1.1 Project Preface

In today's fast-paced urban environment, finding a place to park is a daily challenge. Traffic congestion and a shortage of parking space waste time and fuel, and they frustrate drivers who circle the same streets looking for a vacant spot. Even in smaller cities such as Kolhapur, Ajara and Gadhinglaj, market areas, bus stands, hospitals and temple roads are crowded for most of the day.

At the same time, many people own land, garages or compounds that stand empty during the day. A way for them to offer this space to drivers, and to be paid safely, would help both sides. Parkaro was created as a web-based answer to this problem. It started as our diploma project, which displayed parking locations on a map. In this final year project we have rebuilt it into a working parking marketplace: drivers can find free public parking nearby and reserve and pay for private parking, and parking owners can list their places and manage payments.

The project uses HTML, CSS, JavaScript, PHP and MySQL, together with the open-source Leaflet.js map library and OpenStreetMap, the Razorpay payment gateway, and e-mail, WhatsApp and SMS providers for ticket delivery. It runs on a normal laptop with XAMPP and needs no paid map service.

1.2 Detailed Problem Definition

The following problems were identified in the existing way of finding and paying for parking:

- **Searching wastes time.** There is no single place where a driver can see which parking places exist near them, which are free and which can be reserved.
- **Paper records and cash.** In most small parking lots, vehicle details are written on tokens or in logbooks and payment is in cash. Records are hard to search later, for example after a theft or a dispute, and the owner has no clean record of income.
- **No advance reservation.** A driver cannot be sure that a space will be available when they reach the place.
- **No safe online payment for private owners.** A private owner who wants to rent a space has no simple tool to accept online payment, confirm it correctly, and avoid fraud such as a fake "payment successful" screenshot.
- **No proof of booking.** Without a ticket sent to the customer, there is no clear proof of what was agreed (place, time, vehicle and amount).

Parkaro addresses these problems by combining a map-based search, server-side reservation with a short payment hold, verified online and in-person payment, and automatic ticket delivery.

1.3 Existing System and Its Limitations

The first version of Parkaro (our diploma project) had three separate city pages for Kolhapur, Gadhinglaj and Ajara, a map, and a booking form. During testing and audit we found the following limitations, which motivated this project:

- The price and availability were not controlled by the server; a booking form accepted whatever hours the browser sent.
- There was no real payment step and no definition of when a booking was "paid".
- Parking owners could not register their own places; locations were fixed in the code.
- Each city had its own copy of the booking page, so a fix had to be repeated three times.
- There was no ticket delivery to the customer and no per-user dashboard.
- Some database credentials were repeated in several files.

1.4 Objectives

1. To let drivers find parking in Kolhapur, Ajara and Gadhinglaj on a map, with distance and directions, and to separate free public places from reservable private places.
2. To let parking owners register their places with price, capacity, opening hours and the payment methods they accept.
3. To let a driver reserve a space for a chosen time window, with the price and capacity decided by the server and the space held for a short time while they pay.
4. To accept online payment through Razorpay and in-person payment (cash or UPI paid to the owner), and to confirm a reservation only after the payment has been verified.
5. To send the customer a ticket by e-mail, WhatsApp and SMS after confirmation, once per channel, and to record delivery status and allow retries.
6. To give owners a reservation list and an earnings report that counts only verified payments.
7. To protect users' data and money with sound security practice, and to prove the important rules with automated tests.

1.5 What Is New Compared With the Diploma Version

Area	Diploma version (2024-25)	This version
Parking places	Fixed in code; three city pages	Owners register places; one Find Parking page for all three cities; 138 free public spots as a built-in list
Booking	Simple form, price from the browser	Server-side reservation: validates time window, checks capacity, calculates price, 20-minute payment hold
Payment	None	Razorpay online payment with server-side verification; cash / UPI-to-owner confirmed by the owner
Ticket	Page shown after booking	Signed ticket link, PDF ticket, e-mail / WhatsApp / SMS delivery with status and retry
Users	Basic login	Hashed passwords, CSRF protection, dashboard with status of every reservation
Owners	Zone-based admin pages	Owner portal: own places, payment methods, reservations, earnings
Home and login pages	Dark theme, different styles	One consistent modern design; Live map page with GPS and directions
Testing	Manual	938 automated checks plus a manual XAMPP checklist

CHAPTER – 2: ANALYSIS

2.1 System Analysis

A driver opens the website on a laptop or a phone and registers with a name, e-mail address, car number and password. After logging in, the driver can search for parking by city, area name, or by using the device location. The result shows two kinds of places clearly separated: **bookable** places registered by owners (with price and capacity) and **free** public spots (information only; availability is unknown). For a bookable place the driver enters the arrival and departure time and the car number, and may add a mobile number for WhatsApp or SMS. The server checks the request, calculates the amount and holds the space for 20 minutes.

The driver then pays online, or pays the owner at the place. Only after the payment is verified does the reservation become confirmed; the ticket then appears in the driver's dashboard and is sent by e-mail and, if chosen, by WhatsApp and SMS. The parking owner uses a separate admin area to register places, choose accepted payment methods, see reservations and confirm cash or UPI received, and see earnings.

The design principle throughout is that **the browser is never trusted** for money or availability. Prices, capacity, time limits and payment status are always decided or verified on the server.

2.2 Requirement Specification

2.2.1 Functional requirements

ID	Requirement
FR1	Users can register and log in; owners (admins) log in separately.
FR2	Users can search places by city and text, with optional time window, and use the device location to sort by distance.
FR3	The system shows a map with bookable places, free places and the user's position, and a Google Maps directions link.
FR4	Owners can add, edit and switch off places, and choose online and/or at-the-place payment.
FR5	Users can reserve a space for a time window; the server validates the window, checks capacity and calculates the price.
FR6	A reservation is held for 20 minutes; an unpaid hold expires and releases the space.
FR7	Online payment through Razorpay; confirmation only after server-side verification (callback and webhook).
FR8	Owners can confirm cash or UPI-to-owner payment after reviewing the reservation and ticking that the money was received.
FR9	After confirmation the system sends a ticket by e-mail, WhatsApp and SMS (where configured), once per channel.
FR10	Owners can see the status of each message and retry failed or unsent ones.
FR11	Users can view and download tickets and cancel unpaid reservations.
FR12	Owners can view earnings by day, month and place, counting only verified payments.

2.2.2 Non-functional requirements

- **Security:** hashed passwords, prepared SQL statements, CSRF tokens, signed ticket links, owner-scoped data access, no secrets in the code.
- **Correctness:** no double booking beyond capacity; no double payment; no duplicate messages.
- **Reliability:** a failure of an external provider (SMS, e-mail, gateway) must not corrupt a booking or a payment.
- **Usability:** responsive pages that work on a phone; plain-language messages.
- **Compatibility:** runs on XAMPP (Apache, PHP 8.2, MariaDB 10.4) on Windows; the database upgrade is additive and safe to repeat.

2.2.3 Software requirements

1. Web browser (Chrome, Edge, Firefox)
2. Operating system: Windows 10/11 (development also tested on Linux)
3. XAMPP Control Panel (Apache, PHP 8.2, MariaDB/MySQL)
4. Languages: HTML, CSS, JavaScript, PHP
5. Libraries: Leaflet.js, OpenStreetMap tiles, dompdf (PDF tickets)
6. External services (optional): Razorpay account, SMTP mailbox, Twilio account (SMS and WhatsApp)
7. Database: MySQL / MariaDB

2.2.4 Hardware requirements

1. SSD 256 GB or more
2. RAM 8 GB
3. Processor: Intel Core i3 or above
4. Mobile phone or laptop with a modern browser and (optionally) location access

2.3 Purpose

The purpose of Parkaro is to give drivers and parking owners one simple, safe web platform for finding, reserving and paying for parking. By showing nearby places, reserving a space before arrival, confirming payment reliably and sending a ticket, the system reduces the time spent searching for parking, gives owners a clean digital record of bookings and income, and discourages disorderly street parking.

2.4 Scope

The current scope covers three cities of Kolhapur district (Kolhapur, Ajara and Gadhinglaj), parking for cars, reservation by time window, and payment through Razorpay or at the place. It is suitable for private garages, compounds, markets and small commercial lots. It does not yet use sensors or cameras, so the number of free spaces is a count of reservations, not a live reading of the physical lot.

2.5 Feasibility

- **Technical:** all technologies are mature and open source; the system runs on a student laptop.
- **Economic:** no paid map API is used. Razorpay, SMTP and Twilio charge per use only when switched on, so development and demonstration cost nothing.
- **Operational:** owners and drivers use ordinary web pages; the owner confirmation screen is a single page with two choices and one tick box.

CHAPTER – 3: DESIGN

3.1 Block Diagram

Figure 3.1 shows the architecture. Drivers and owners use a browser. The PHP pages pass through a security layer (sessions, CSRF checks, signed links) to the reservation, payment, notification and map modules. All modules store data in MySQL/MariaDB. External services are reached only from the server (Razorpay, SMTP, Twilio) or directly from the browser (map tiles and Google Maps directions links).

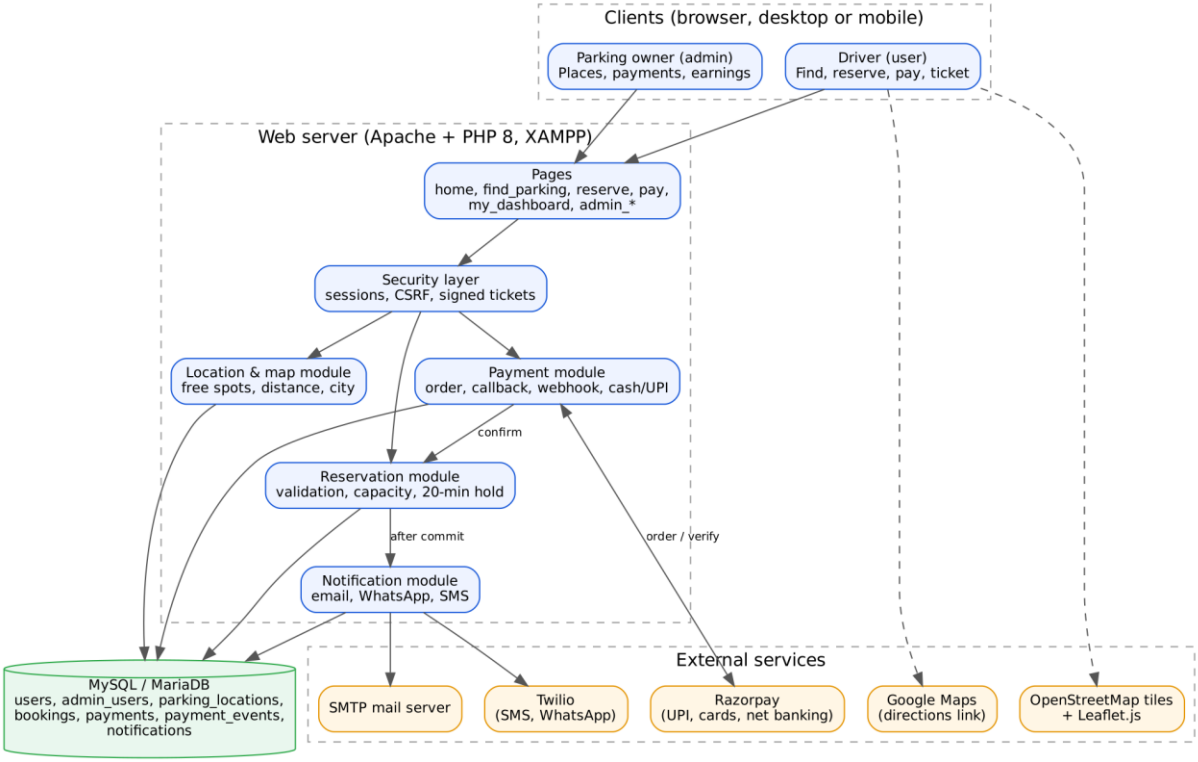


Fig. 3.1 Block diagram of the Parkaro system

3.2 Flowchart

Figure 3.2 shows the life of a reservation. The important rule is that the “confirmed” state is reached only through a verified payment, and the ticket message is sent only after that.

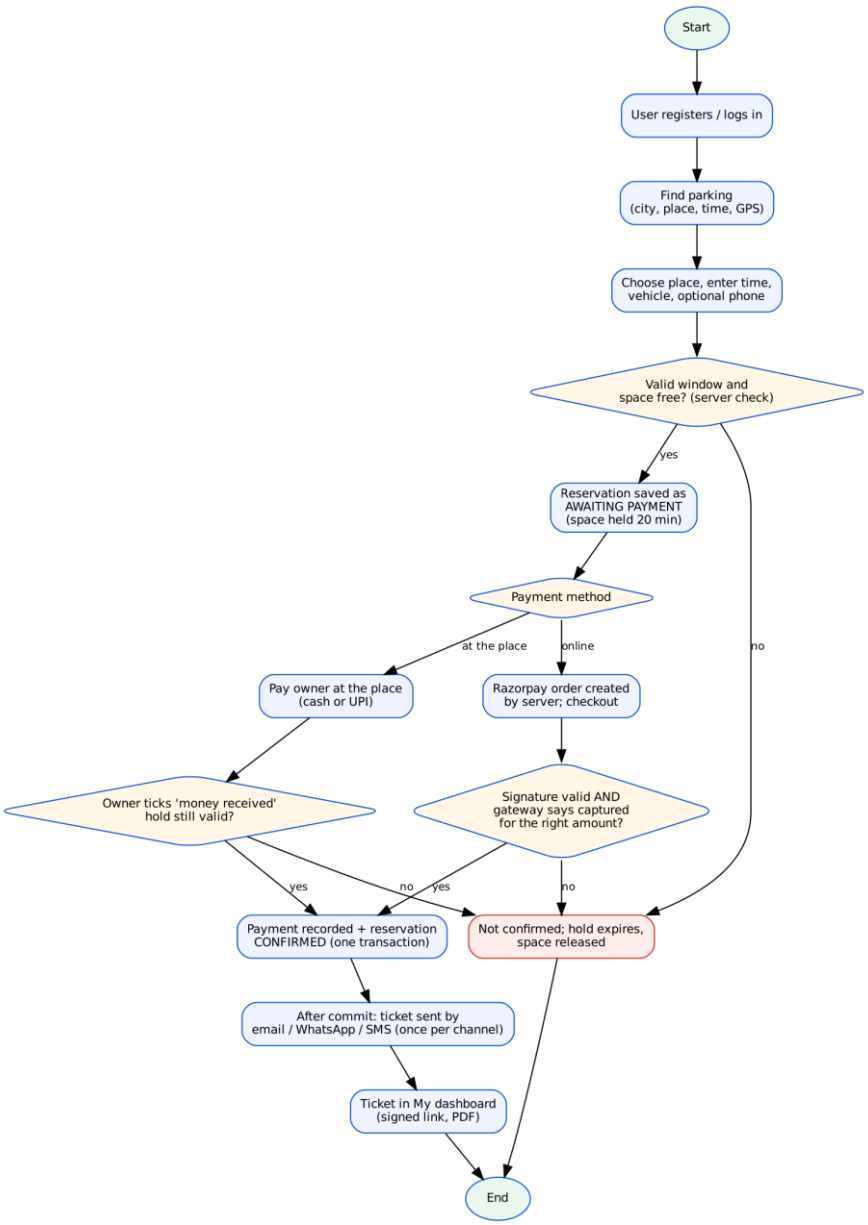


Fig. 3.2 Flowchart: reservation, payment, confirmation and ticket

3.3 Use Case Diagram

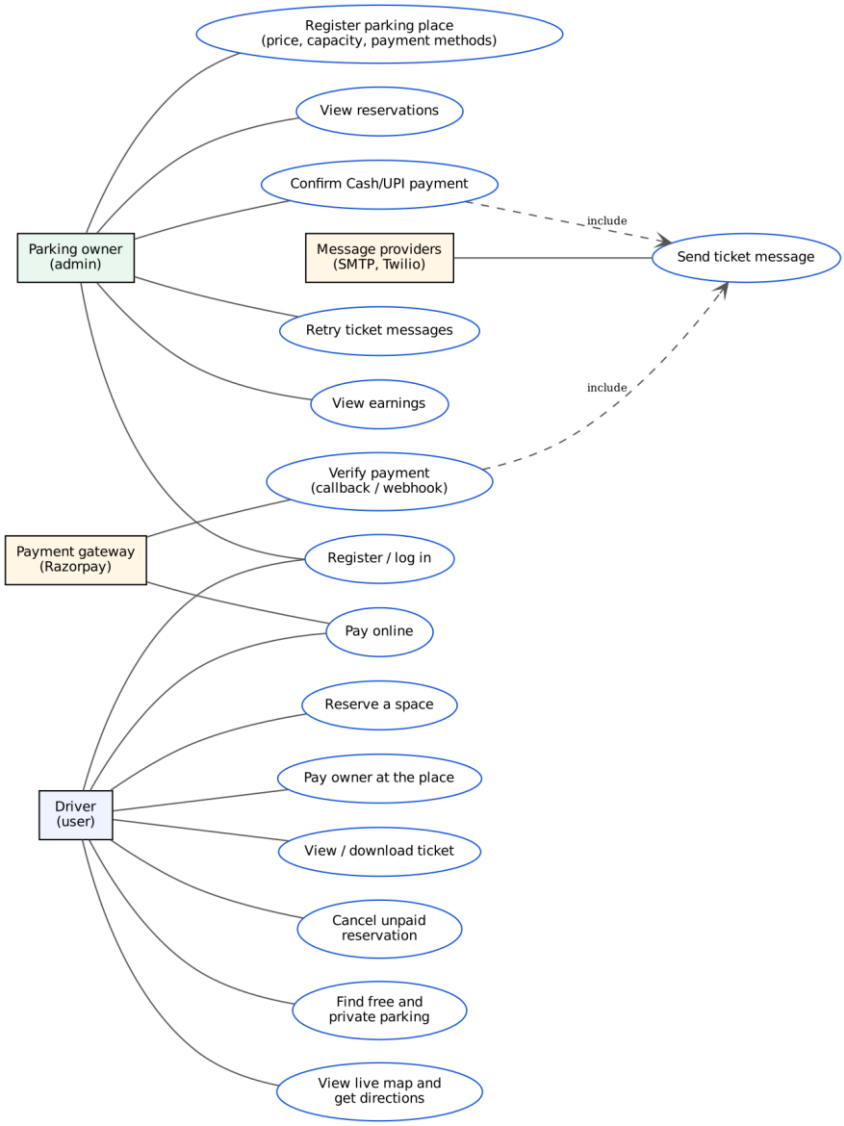


Fig. 3.3 Use case diagram

3.4 Reservation and Payment States

State	Meaning
pending_payment	Reservation saved; space held until hold_expires_at (20 minutes). No payment yet. Not counted as income.
confirmed	A payment has been verified (gateway) or recorded by the owner (cash / UPI). The ticket is available and the message is sent.
expired	The hold ran out without payment. The space is released.
cancelled	The user cancelled an unpaid reservation. Paid reservations cannot be cancelled in this version (no refunds implemented).

Payment record	How it is created	Label shown
razorpay / gateway_api	Checkout signature valid AND the gateway reports “captured” for the stored amount	Verified online payment

Payment record	How it is created	Label shown
razorpay / gateway_webhook	Webhook with valid signature reports “captured”	Verified online payment
cash / admin_cash	Owner confirms cash received (tick box, review screen)	Cash paid at the place
upi_owner / admin_upi	Owner confirms UPI paid directly to the owner	UPI received by owner

3.5 Ticket Message Flow

Figure 3.4 shows how a message is sent after confirmation. The database transaction is committed first; only then is the provider called, so that no external call is made while a transaction is open. A row in the notifications table, unique for each booking and channel, is inserted before sending. Whoever inserts it first sends; a duplicate callback or a second click finds the row and sends nothing.

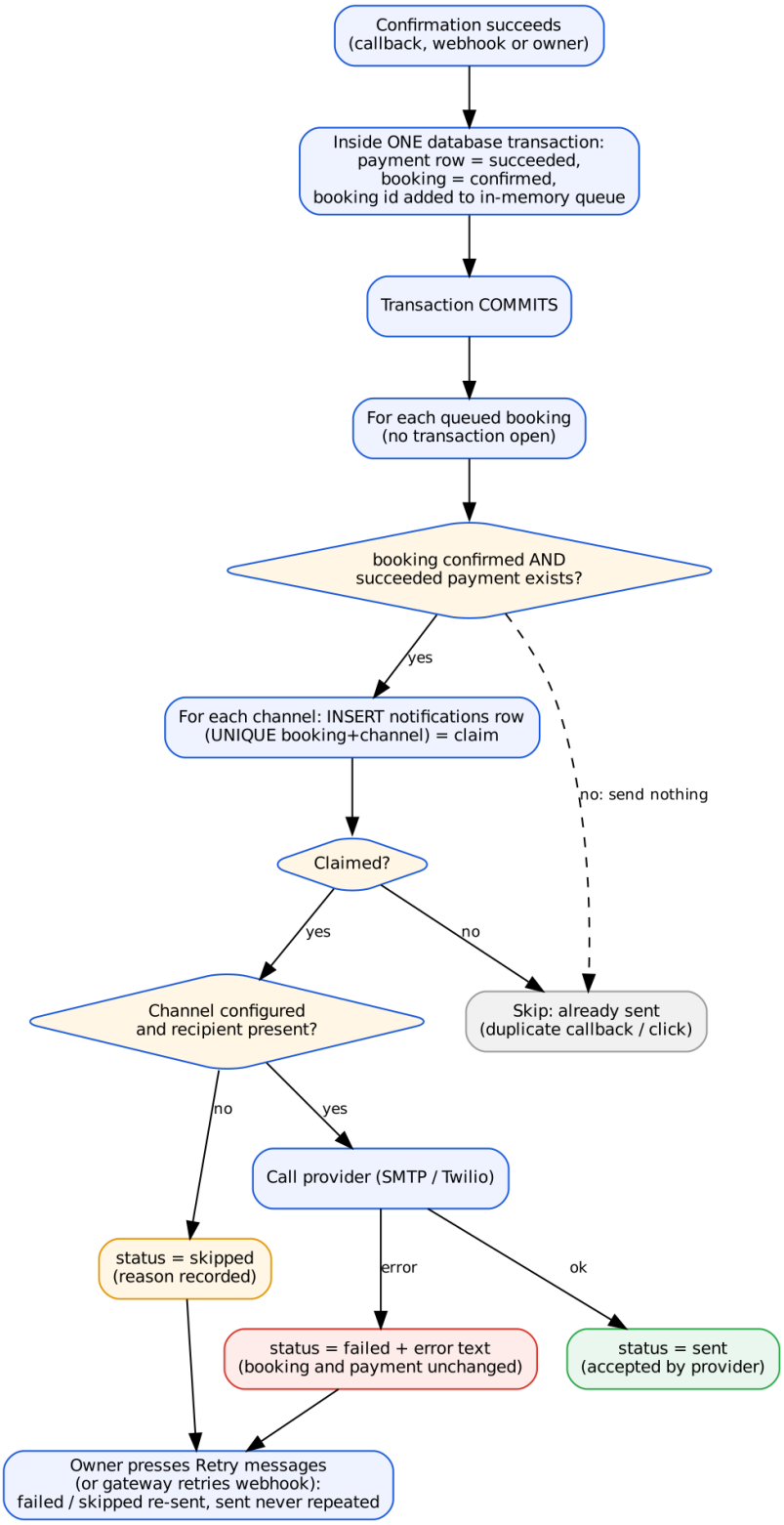


Fig. 3.4 Notification delivery flow

CHAPTER – 4: SYSTEM MODELING

4.1 Data Flow Diagram

Level 0 (context diagram)

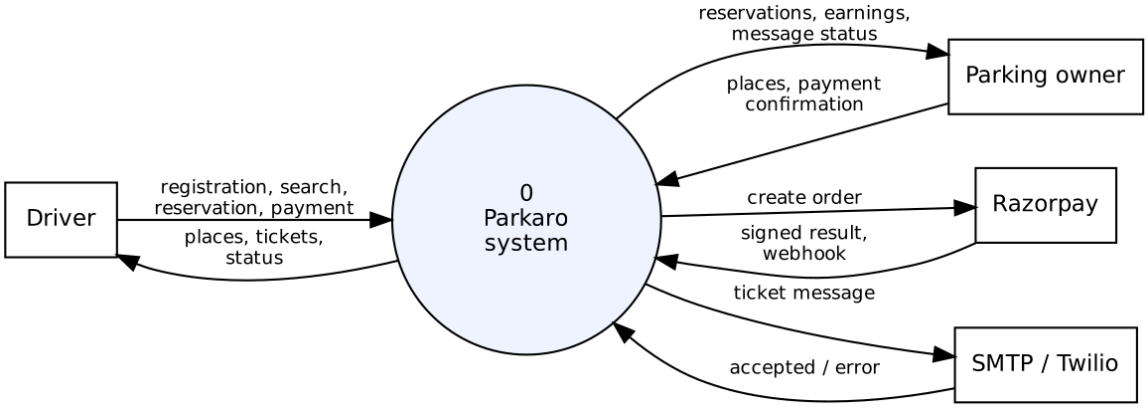


Fig. 4.1 DFD level 0

Level 1

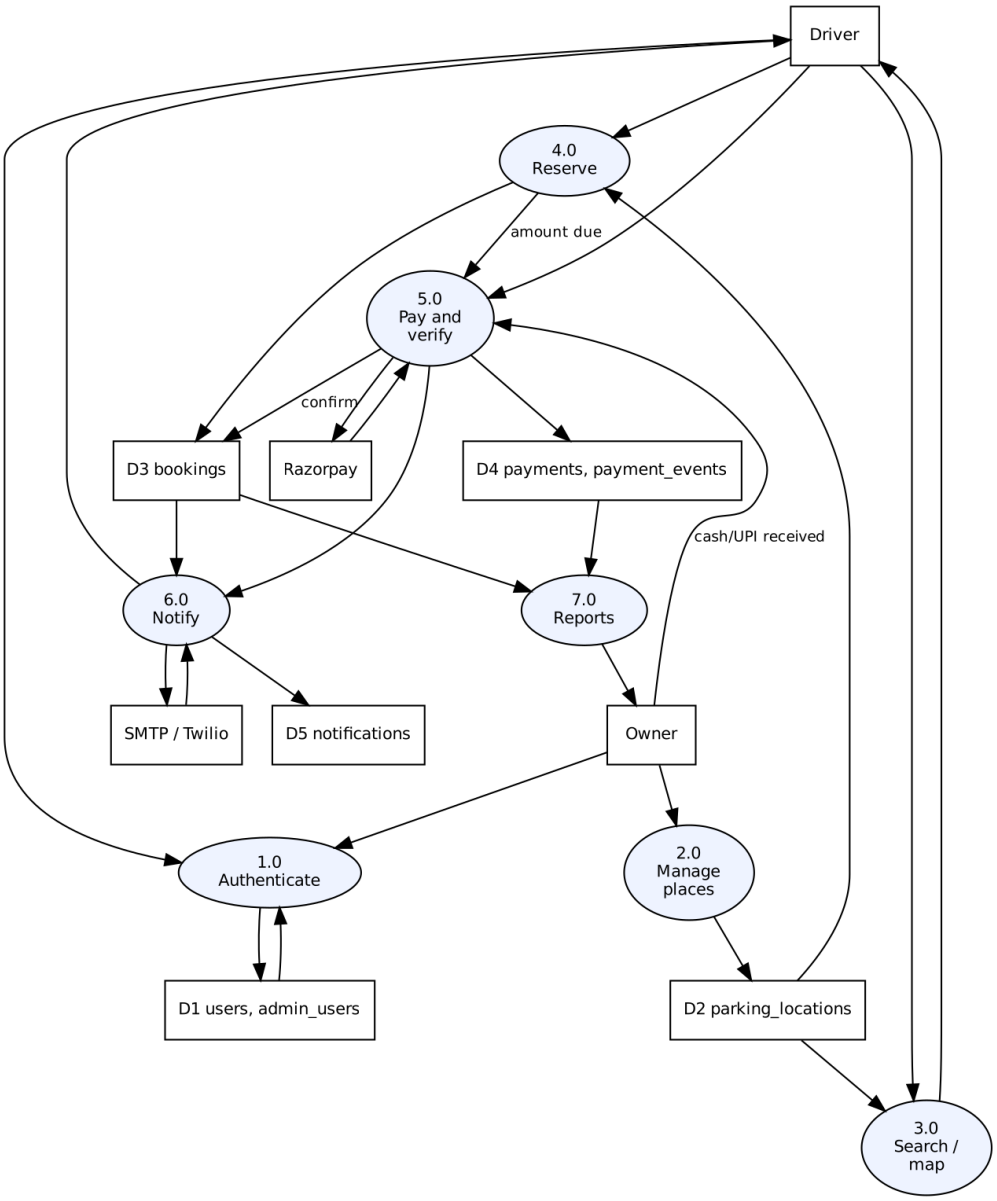


Fig. 4.2 DFD level 1

4.2 ER Diagram

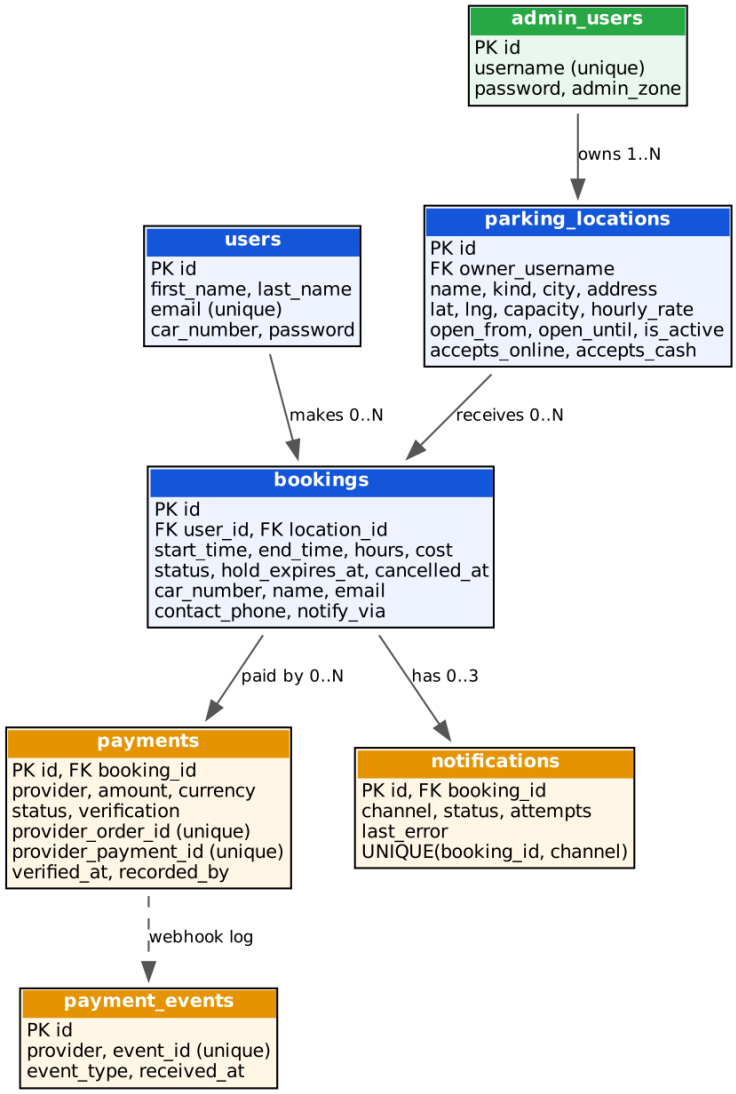


Fig. 4.3 Entity relationship diagram

4.3 Data Dictionary (main tables)

parking_locations

Column	Type	Description
id	INT, PK	Place number
owner_username	VARCHAR(50)	The admin who owns the place; used to restrict every owner action
name, kind, city, address	VARCHAR	Name, public / private, city and address
lat, lng	DECIMAL(9,6)	Map position (optional)
capacity, hourly_rate	INT	Number of cars at the same time; price per started hour (rupees)
open_from, open_until	TIME	Opening hours; empty = 24 hours
is_active	TINYINT(1)	Switched on / off

Column	Type	Description
accepts_online, accepts_cash	TINYINT(1)	Payment methods accepted by this place (default 1, 1)

bookings (reservations)

Column	Type	Description
id	INT, PK	Reservation number
user_id, location_id	INT, FK	Who reserved which place
start_time, end_time, hours, cost	DATETIME / INT	Time window; billed hours; amount calculated by the server
status	VARCHAR(20)	pending_payment, confirmed, expired, cancelled
hold_expires_at	DATETIME	End of the 20-minute payment hold
contact_phone, notify_via	VARCHAR	Mobile number and chosen channels for WhatsApp / SMS

payments

Column	Type	Description
booking_id	INT, FK	The reservation paid for
provider	VARCHAR(20)	razorpay, cash or upi_owner
provider_order_id / provider_payment_id	VARCHAR(64), UNIQUE	Gateway ids; uniqueness prevents the same payment being recorded twice
amount, currency	INT, CHAR(3)	Amount in rupees (taken from the stored reservation)
status	VARCHAR(20)	created, succeeded, failed, refunded
verification	VARCHAR(30)	gateway_api, gateway_webhook, admin_cash, admin_upi
verified_at, recorded_by	DATETIME, VARCHAR	When, and (for owner confirmations) by whom

notifications

Column	Type	Description
booking_id, channel	UNIQUE together	One row per booking and channel (email, whatsapp, sms): guarantees once-only sending
status	VARCHAR(12)	sending, sent, failed, skipped
attempts, last_error	INT, VARCHAR(255)	Number of tries and the short reason of the last failure (no secrets)

CHAPTER – 5: RELATIVE THEORY

5.1 Technologies Used

HTML and CSS

HTML (HyperText Markup Language) structures every page: forms, tables, navigation and map containers. CSS (Cascading Style Sheets) controls layout, colours, fonts and responsiveness. In this version one shared stylesheet (site.css) gives the home, login, registration and booking pages a single consistent design that adapts to phone screens using flexbox and media queries.

JavaScript, Leaflet.js and OpenStreetMap

JavaScript adds interaction in the browser. Leaflet.js is a light open-source library for interactive maps, and OpenStreetMap supplies free map tiles, so no paid map key is needed. The browser Geolocation API provides the driver's position, but only after the user gives permission. The Live map page shows markers for bookable and free places and the user's position.

Distance calculation (Haversine formula)

To sort places by distance the server and the browser use the great-circle (Haversine) formula, which gives the straight-line distance between two latitude/longitude points on a sphere: $d = 2R \cdot \operatorname{asin}(\sqrt{(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2))})$, with $R = 6371$ km. The system shows these as approximate distances and clearly says they are not road distances; for real routes it opens a Google Maps directions link.

PHP

PHP is the server-side language that contains all business rules. It validates input, talks to the database, creates payment orders, verifies payments and sends messages. The older pages use the mysqli extension; the new reservation, payment and notification modules use PDO with prepared statements.

MySQL / MariaDB

A relational database stores users, places, reservations, payments and message logs. Three database features are important in this project: (1) **transactions**, so that a payment record and the reservation status change together or not at all; (2) **row locks** (SELECT … FOR UPDATE) on a parking place, so two simultaneous reservations cannot both take the last space; and (3) **unique constraints**, so the same gateway payment or the same message cannot be stored twice. Schema changes are made by additive migration scripts that can be run more than once safely.

Razorpay and signature verification

Razorpay is a payment gateway that supports UPI, cards and net banking in India. The server creates an order for the amount stored in the reservation. After checkout the browser is sent back with a payment id, order id and signature. A browser result is never trusted by itself: the server recomputes the signature with HMAC-SHA256 using the secret key, then asks Razorpay's API for the real payment and checks that it is "captured" for the right amount. A signed webhook from Razorpay is a second, independent route and is de-duplicated by event id.

E-mail (SMTP) and Twilio

SMTP is the standard protocol for sending e-mail; the project contains a small SMTP client that supports STARTTLS/SSL and login. Twilio's REST API is used for SMS and for WhatsApp messages. All credentials are read from environment variables in a .env file and are never written in the code.

Web security concepts used

- **Prepared statements** keep user text separate from SQL and prevent SQL injection.
- **Password hashing** (password_hash / password_verify) stores only a one-way hash of each password.
- **CSRF tokens** make sure a state-changing request really came from the logged-in user's own page.
- **Signed ticket links** (an HMAC of the booking id) mean a ticket cannot be opened by guessing another booking number.
- **Idempotency** means repeating a request (a retried webhook, a double click) has the same effect as doing it once.

XAMPP

XAMPP bundles Apache, PHP and MariaDB so the whole system can run on a Windows laptop for development and demonstration.

CHAPTER – 6: IMPLEMENTATION

This chapter describes the main modules and the most important parts of the code. The complete source is in the project folder; only short excerpts are shown here.

6.1 Project Structure

File / folder	Purpose
home.html, site.css	Home page and the shared design
index.php, register.php, login1.php	User login, registration and admin login
find_parking.php, booking.html (Live map), parking_points.php	Search, map, nearby places, public map data
reserve.php, pay.php, my_dashboard.php	Reservation form, payment page, user dashboard
payment_callback.php, payment_webhook.php	Razorpay return and webhook endpoints
admin_locations.php, admin_reservations.php, admin_earnings.php	Owner portal
booking_confirmed.php, generate_ticket.php	Ticket page and PDF ticket (signed links)
src/reservations.php	Reservation rules, capacity, holds, confirmation
src/payments.php	Gateway, verification, webhook, cash / UPI confirmation
src/notify.php	Message text, delivery, status, retry, SMTP and Twilio transport
src/locations.php, src/free_parking.php	Place validation; free spots, city and distance helpers
database/migrations/, tools/	Additive migrations and checking tools
tests/	Automated tests (SQLite twin, unit, HTTP, browser)

6.2 Reservation Module

create_reservation() validates the arrival and departure time (minimum 30 minutes, maximum 24 hours, up to 30 days ahead, inside the opening hours), locks the place row, counts how many other reservations overlap the window, and rejects the request if capacity would be exceeded. The amount is calculated as started hours multiplied by the hourly rate. The reservation is then stored as pending_payment with a 20-minute hold.

6.3 Single Confirmation Point

All confirmations — gateway callback, webhook and owner confirmation — go through one function. If the hold has run out it confirms only if the space is still free. After the status is updated, the booking is added to an in-memory queue; the message is sent only after the transaction commits.

```
function confirm_booking_in_tx(PDO $pdo, array $booking, DateTimeImmutable $now): bool
{
    if ($booking['status'] === BK_CONFIRMED) {
        notify_queue((int) $booking['id']);   // sent after commit; once-only by the
notifications table
        return true;
    }
    ...  // hold lapsed? confirm only if the space is still free (row lock on the place)
    q_run($pdo, "UPDATE bookings SET status = 'confirmed', hold_expires_at = NULL,
                 cancelled_at = NULL WHERE id = ?", [(int) $booking['id']]);
    notify_queue((int) $booking['id']);
    return true;
}
```

6.4 Online Payment Module

start_online_payment() creates the Razorpay order for the stored amount. complete_checkout_callback() verifies the signature with HMAC-SHA256, fetches the payment from the gateway, and checks status and amount before recording the payment as succeeded and confirming the reservation in one transaction. handle_gateway_webhook() does the same for webhooks and also stores the event id in payment_events so a repeated event is ignored. If the gateway is not configured, the pay page does not show an online payment button at all.

6.5 Cash and UPI Confirmation by the Owner

The owner opens “Confirm Cash/UPI Payment” for an unpaid reservation. The review screen shows the reservation number, customer, place, vehicle, time window and amount. The owner chooses Cash or UPI received by owner and must tick that the money was really received. The server then re-checks everything under a lock:

```
// record_cash_payment_tx() - essential guards (trimmed)
$loc = lock_location($pdo, (int) $b['location_id']);
if ($loc === null || $loc['owner_username'] !== $adminUsername) return
['ok'=>false,'error'=>'Reservation not found.'];
if (payment already succeeded)                  return ['ok'=>false,'error'=>'A payment is
already recorded...'];
if (status is cancelled)                        return ['ok'=>false, ...];
if (status != pending || hold_expires_at <= now) return ['ok'=>false,'error'=>'The hold on this
reservation has ended...'];
if ($loc['accepts_cash'] != 1)                  return ['ok'=>false,'error'=>'This place is set
to online payments only...'];
if ($expectedAmount !== null && $expectedAmount !== (int) $b['cost']) return
['ok'=>false,'error'=>'The amount does not match...'];
INSERT INTO payments (... provider 'cash' | 'upi_owner', amount = stored cost, recorded_by =
owner ...)
confirm_booking_in_tx(...)
```

The amount always comes from the stored reservation. The amount posted by the form is only a guard: if it differs, nothing is recorded. An owner of a different place sees only “Reservation not found”.

6.6 Notification Module

deliver_booking_confirmation() first checks that the booking is confirmed and has a succeeded payment. For each requested channel it inserts a row in the notifications table; the UNIQUE

(booking_id, channel) key makes the insert succeed only once, so only one process sends. The result is stored as sent, failed (with a short reason) or skipped (channel not configured or no recipient). A failure never throws and never touches the booking or payment. Retry re-sends only failed or skipped channels, never sent ones.

```
INSERT INTO notifications (booking_id, channel, status, attempts, created_at, updated_at)
VALUES (?, ?, 'sending', 0, ?, ?);              -- whoever inserts first sends
-- duplicate key (23000): only a 'failed' row with attempts < 3 may be re-claimed
UPDATE notifications SET status = 'sending' WHERE booking_id = ? AND channel = ?
   AND status = 'failed' AND attempts < 3;
```

"Sent" means the provider accepted the message. The system never claims that a message was delivered to a phone or read, because the provider's acceptance is the only evidence available.

6.7 Find Parking, Map and City Handling

find_parking.php lists bookable places (from the database) and free spots (a built-in list of 138 spots). A place's city is decided by its map position first, then its city text, then its address, so a place in Ajara that the owner typed as "Kolhapur" is still found under Ajara. Free spots that duplicate a bookable place are hidden. The directions link uses the user's real position only if the browser provided it.

```
function fp_distance_m(float $lat1, float $lng1, float $lat2, float $lng2): float
{
    $r = 6371000.0;
    $p1 = deg2rad($lat1); $p2 = deg2rad($lat2);
    $dp = $p2 - $p1;      $dl = deg2rad($lng2 - $lng1);
    $a = sin($dp / 2) ** 2 + cos($p1) * cos($p2) * sin($dl / 2) ** 2;
    return 2 * $r * asin(min(1.0, sqrt($a)));
}
```

6.8 Database Migrations

Migration 001 adds places, payments and the new reservation columns; migration 002 adds the payment-method columns, the phone columns and the notifications table. Both use CREATE TABLE IF NOT EXISTS and information_schema checks, so they can be run repeatedly, never drop or delete data, and work on MariaDB 10.4. The pages continue to work (without messages) if migration 002 has not been applied. tools/check_migration.php reports what is applied, and tools/apply_migration.php applies a migration after a confirmation flag.

6.9 Configuration

Settings are read from a .env file that is not part of the code or the ZIP: database, application key, Razorpay keys and webhook secret, SMTP, Twilio and the public base URL (see Appendix B). If a setting is missing, the matching channel is simply off and the page says so.

CHAPTER – 7: TESTING

7.1 Testing Approach

Because this system handles money and bookings, the important rules were written as automated tests. The tests use a SQLite copy of the database schema, a fake Razorpay server, and fake e-mail/SMS transports that record what would have been sent. A local fake SMTP server tests the real SMTP client, and the Twilio request format is checked against a faked HTTP layer. Real browser tests (headless Chromium) check the home page and the Live map page.

7.2 Test Results

Test suite	Checks passed	What it covers
tests/sqlite/run.php (rules)	228 / 228	Reservation, capacity, holds, payment verification, webhooks, cash and UPI, earnings
tests/sqlite/pages.php (real pages)	224 / 224	Pages over HTTP: owner permissions, payment methods, messages, retries, ticket access
tests/sqlite/auth.php	197 / 197	Login, registration, sessions, CSRF, admin access
tests/unit/run.php	69 / 69	Validation and helper functions
tests/unit/ticket_test.php	5 / 5	Signed ticket tokens
tests/unit/free_parking_test.php	49 / 49	Cities, distances, directions links, free spots
tests/unit/notify_test.php	44 / 44	Phone numbers, Twilio requests, SMTP against a fake server, escaping
tests/http/run.php	58 / 58	HTTP behaviour and security headers
tests/browser/geolocation.mjs	22 / 22	Live map and location permission
tests/browser/pages.mjs	42 / 42	Home page and Live map page in a browser
Total	938 / 938	All passing at the time of writing; php -l reported no syntax errors

7.3 Important Scenarios Tested

- An owner cannot change the payment methods or confirm a payment of another owner's place.
- A forced online payment at a cash-only place, and a forced cash confirmation at an online-only place, are refused on the server.

- A forged or unverified payment, a failed payment, and a payment that is only "authorized" confirm nothing and send nothing.
- Repeated callbacks and duplicate webhooks create one payment and send one message per channel.
- A wrong amount posted with the owner confirmation, a missing "money received" tick, an invented payment method, and an expired hold are all refused and record nothing.
- A second confirmation click is refused; no second message is sent.
- When the e-mail or SMS provider fails, the booking and payment still stand, the failure and its reason are recorded, and Retry sends only the missing channels.
- A ticket cannot be opened without its signed token, or with another booking's id.
- Earnings count each verified payment once and never count pending or expired reservations.

7.4 What Was Not Tested

The following need real accounts or infrastructure and were **not** tested in the automated suites; they should be tested by the team before a real deployment (the checklist is in Appendix C):

- Real Razorpay payments and webhooks (the tests use a fake gateway).
- Real e-mail delivery through an SMTP server, including STARTTLS/SSL.
- Real SMS and WhatsApp delivery through Twilio, including WhatsApp template approval.
- The automated MySQL migration test (tests/integration/mysql_migration_test.php) is written but needs a real MySQL/MariaDB server and a throw-away database.
- Real map tiles, GPS accuracy and Google Maps in a live browser.

CHAPTER – 8: OUTPUT

The following screenshots were taken from the running application with demonstration data (names and places are examples).

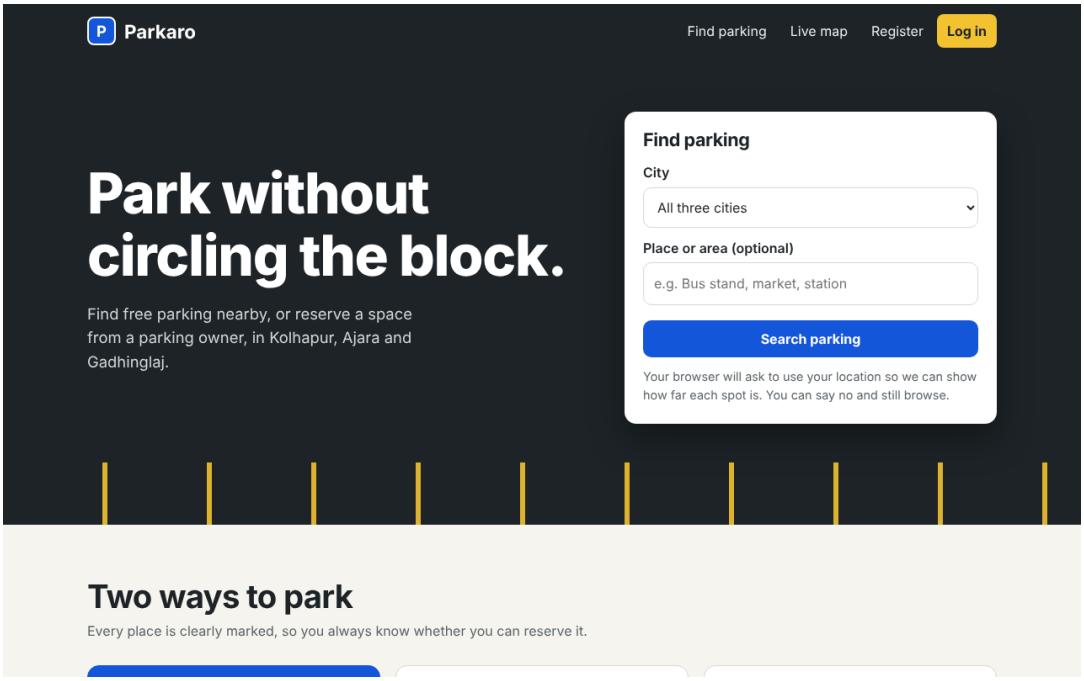


Fig. 8.1 Home page

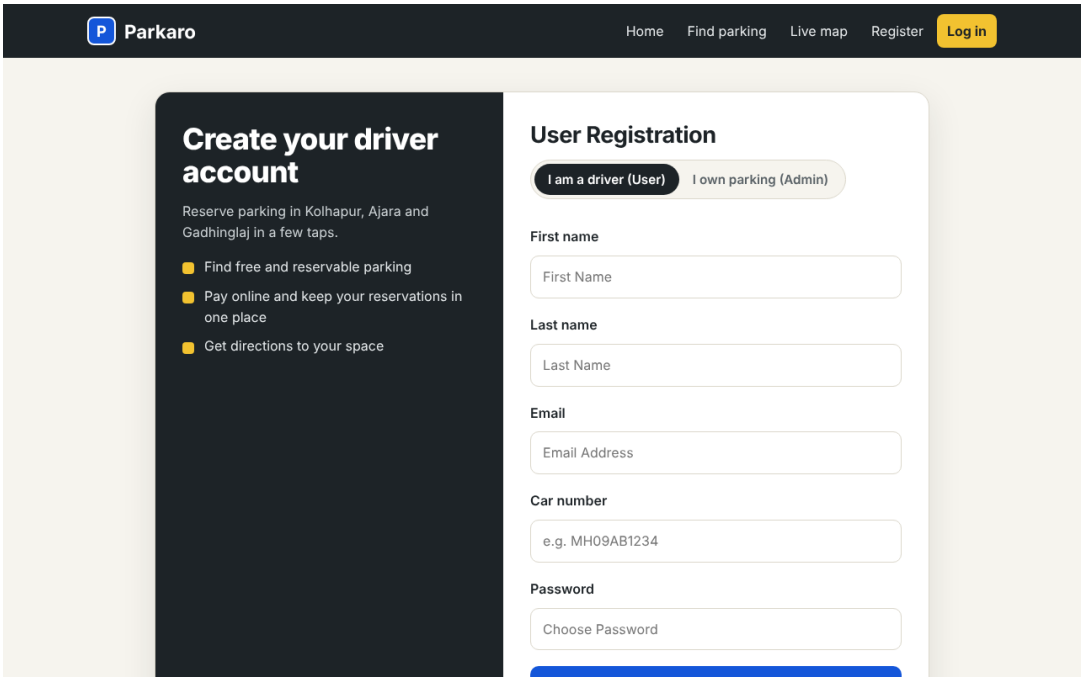


Fig. 8.2 User registration

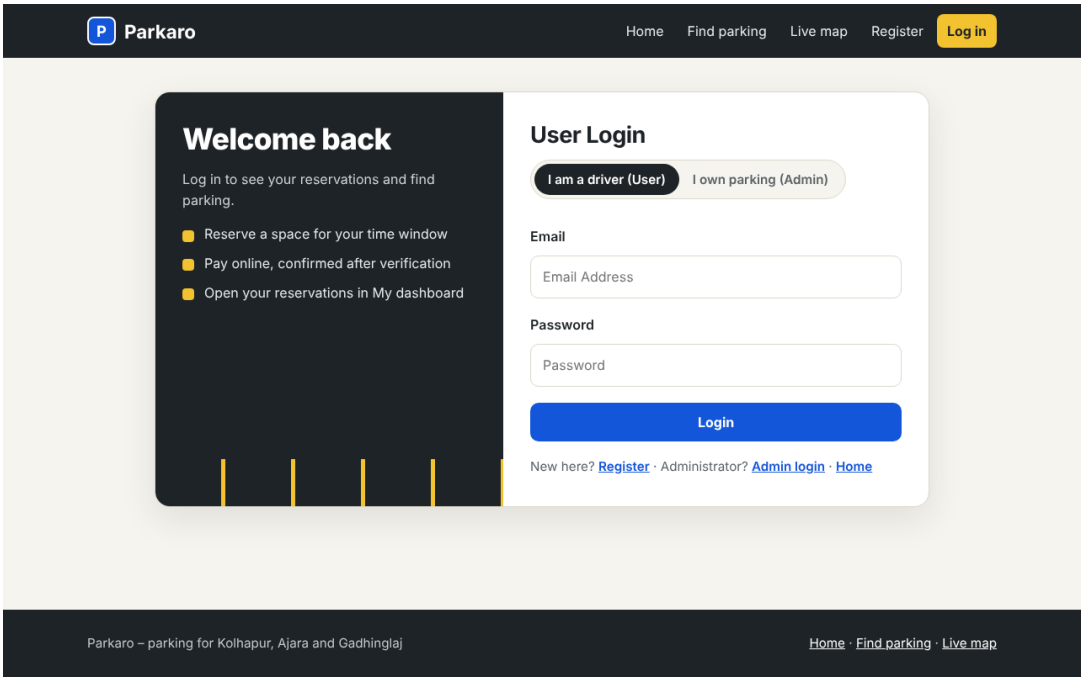


Fig. 8.3 User login

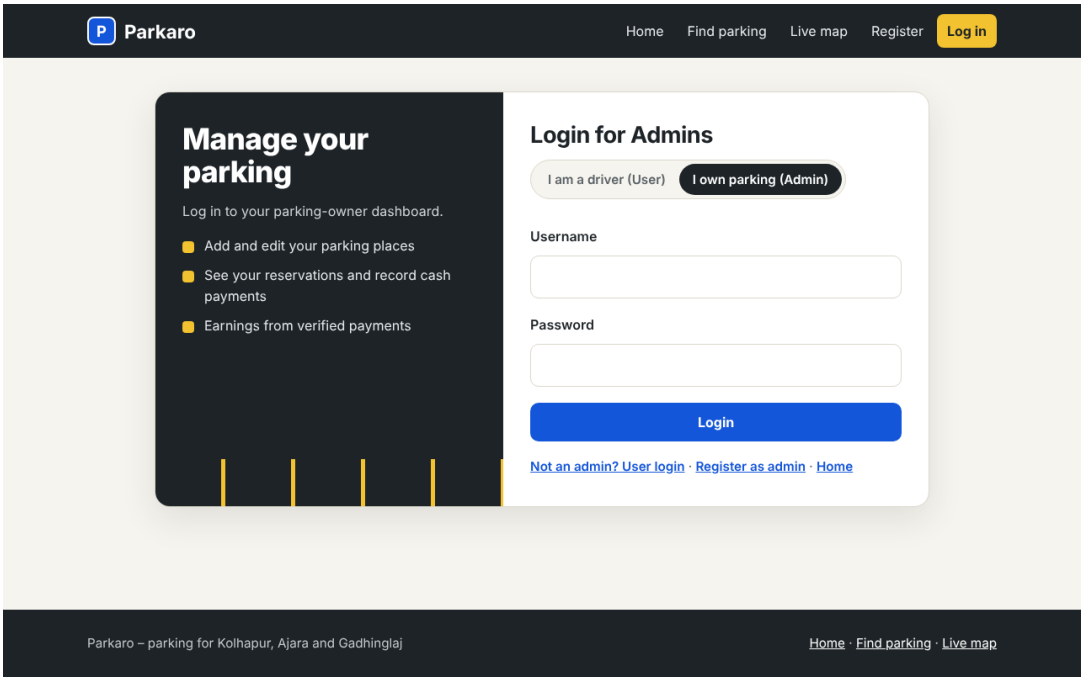


Fig. 8.4 Parking owner (admin) login

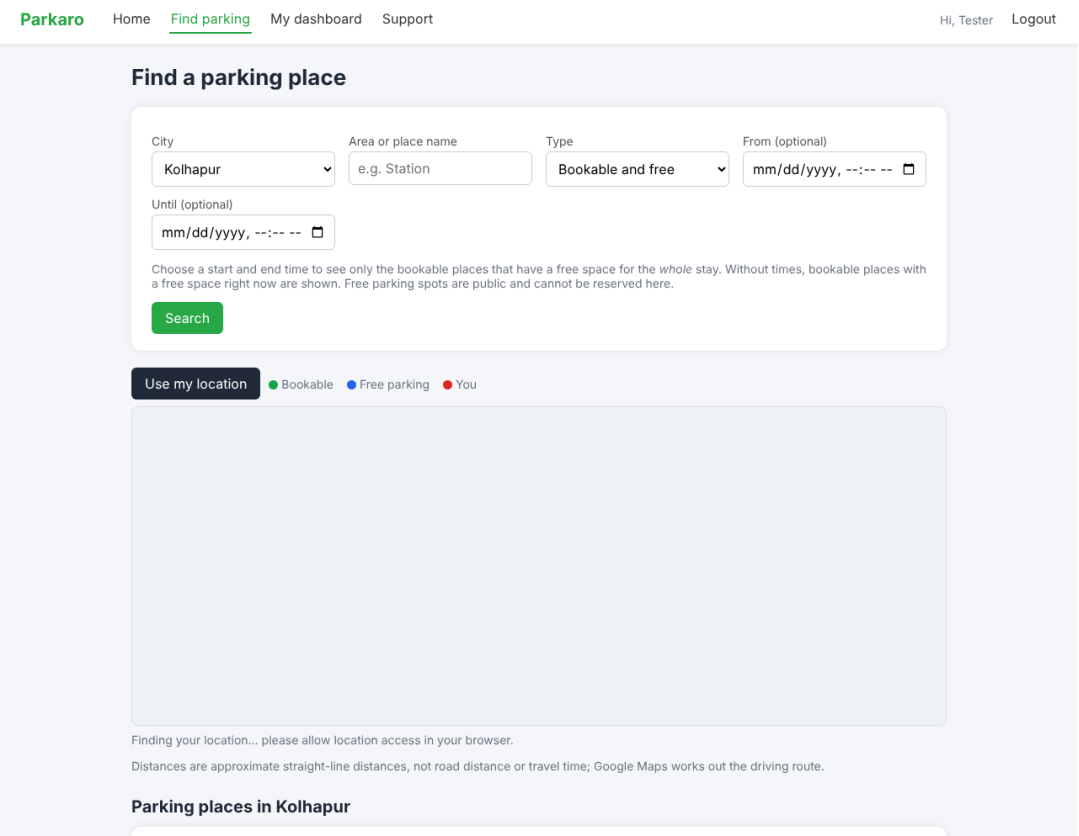


Fig. 8.5 Find a parking place (the map tiles are blank in this offline capture)

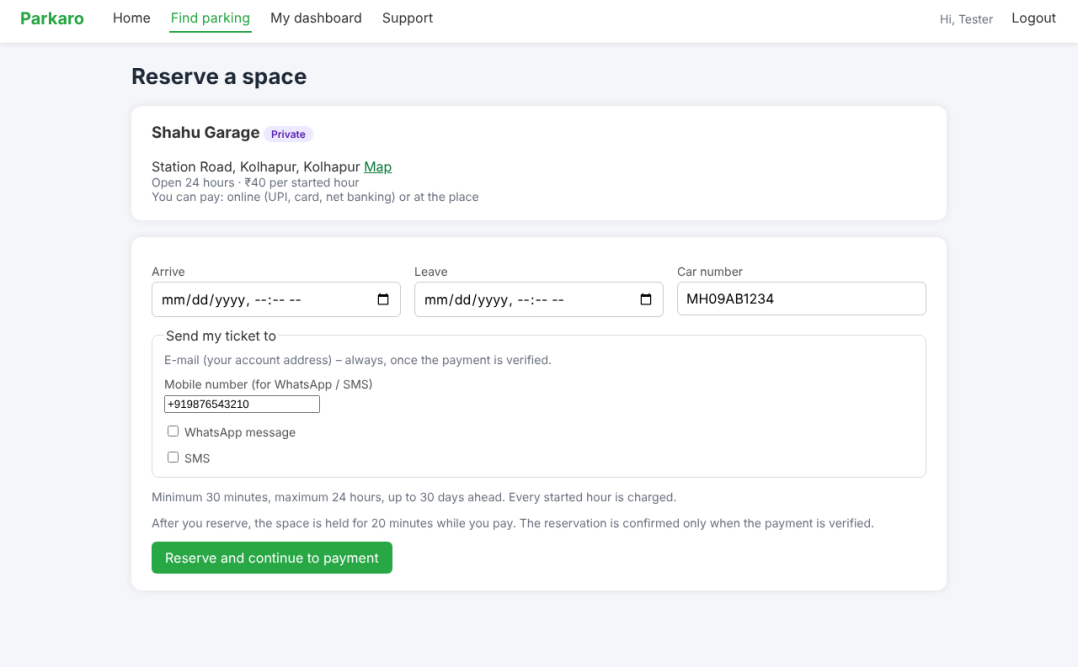


Fig. 8.6 Reserve a space, with phone number and message choices

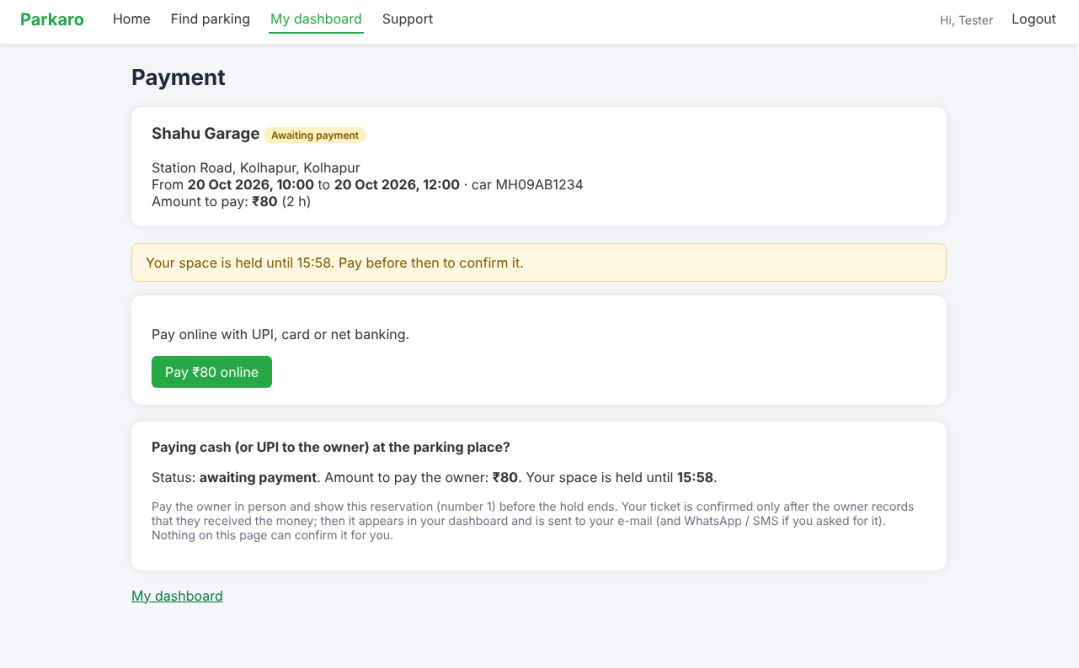


Fig. 8.7 Payment page: awaiting payment, hold time, online or pay the owner

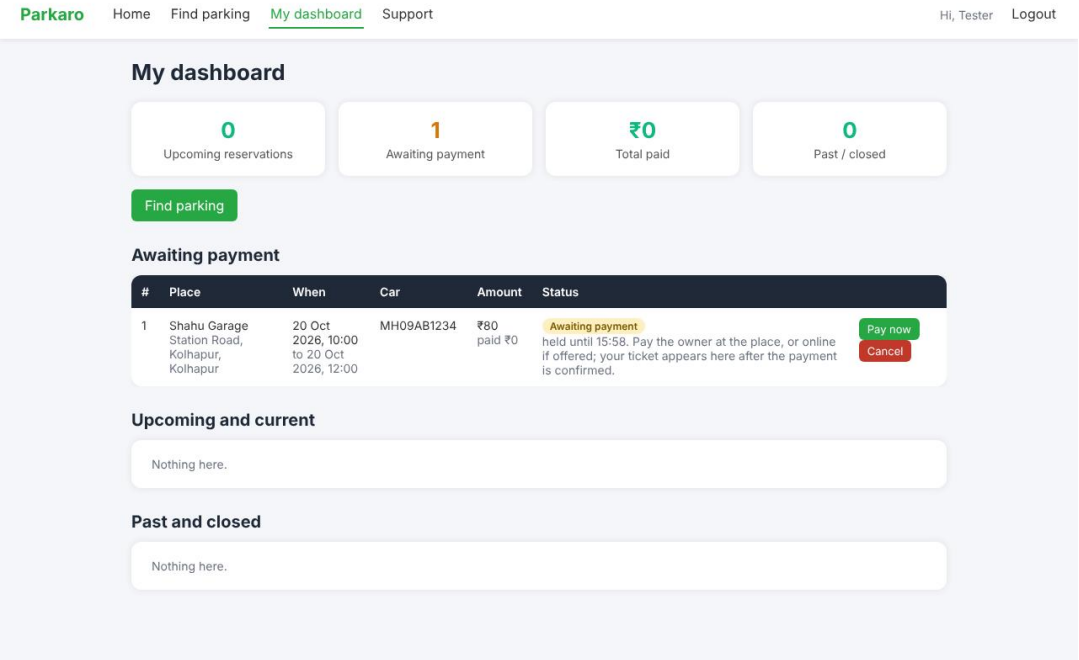


Fig. 8.8 User dashboard with a reservation awaiting payment

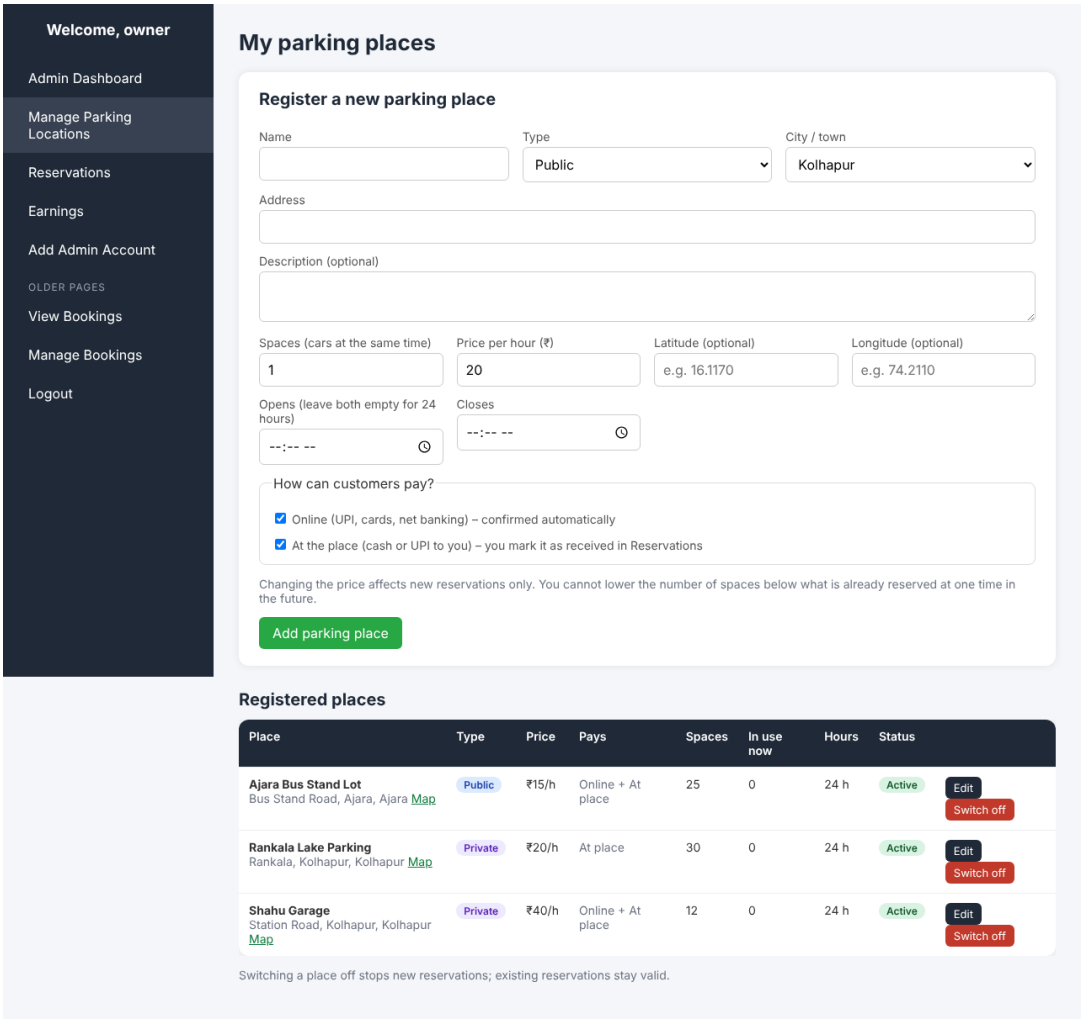


Fig. 8.9 Owner: register a parking place and choose payment methods

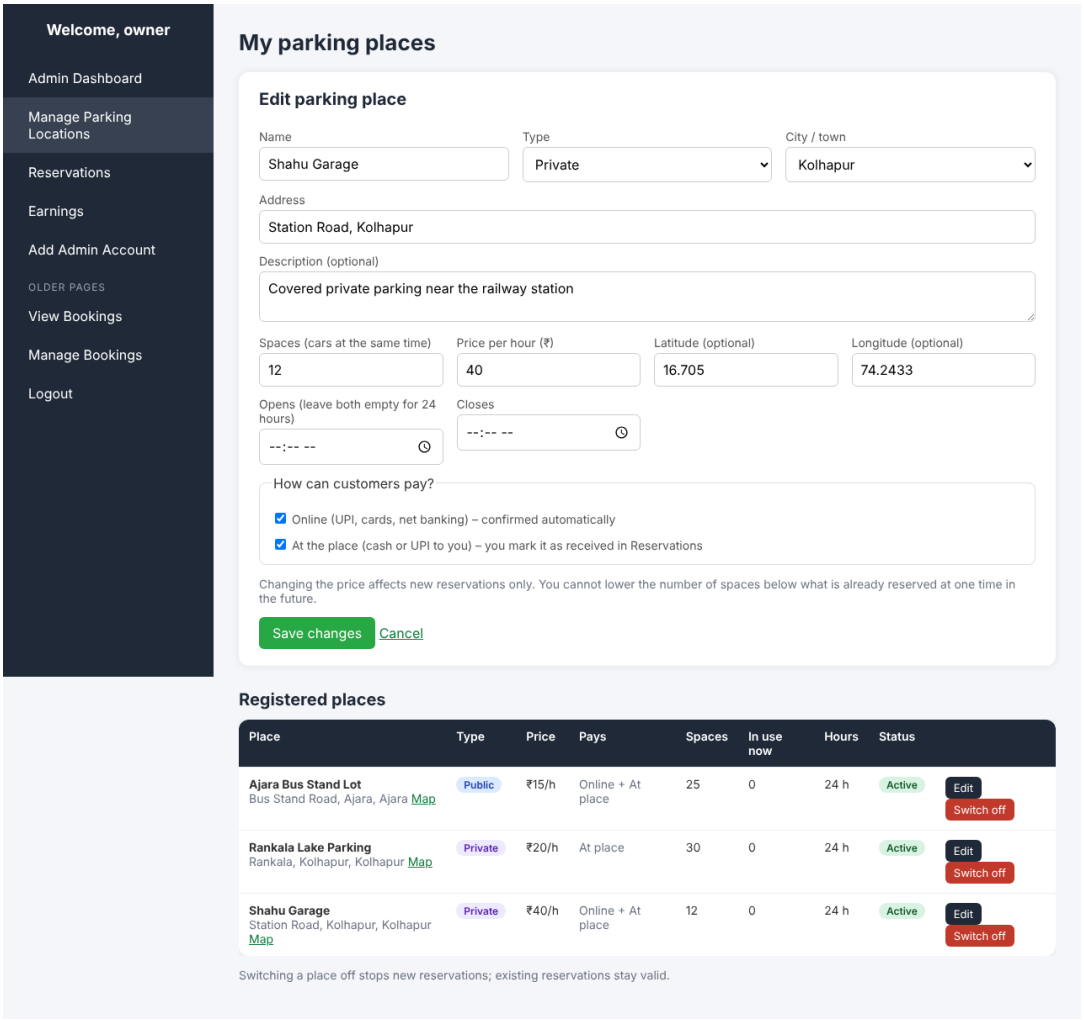


Fig. 8.10 Owner: edit a place

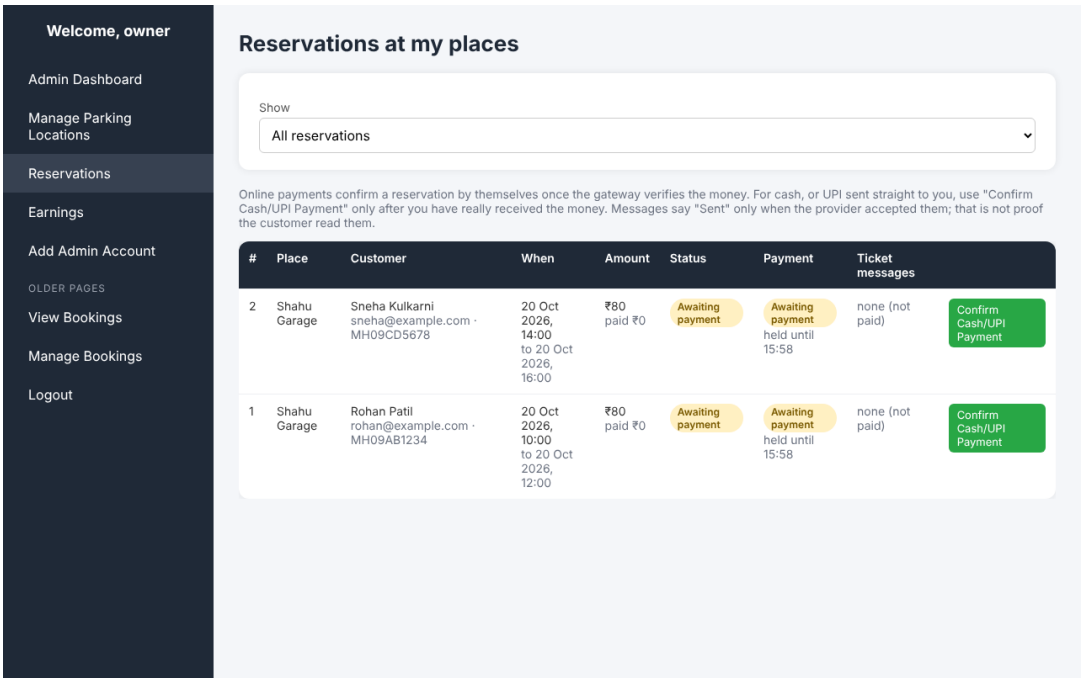


Fig. 8.11 Owner: reservations awaiting payment

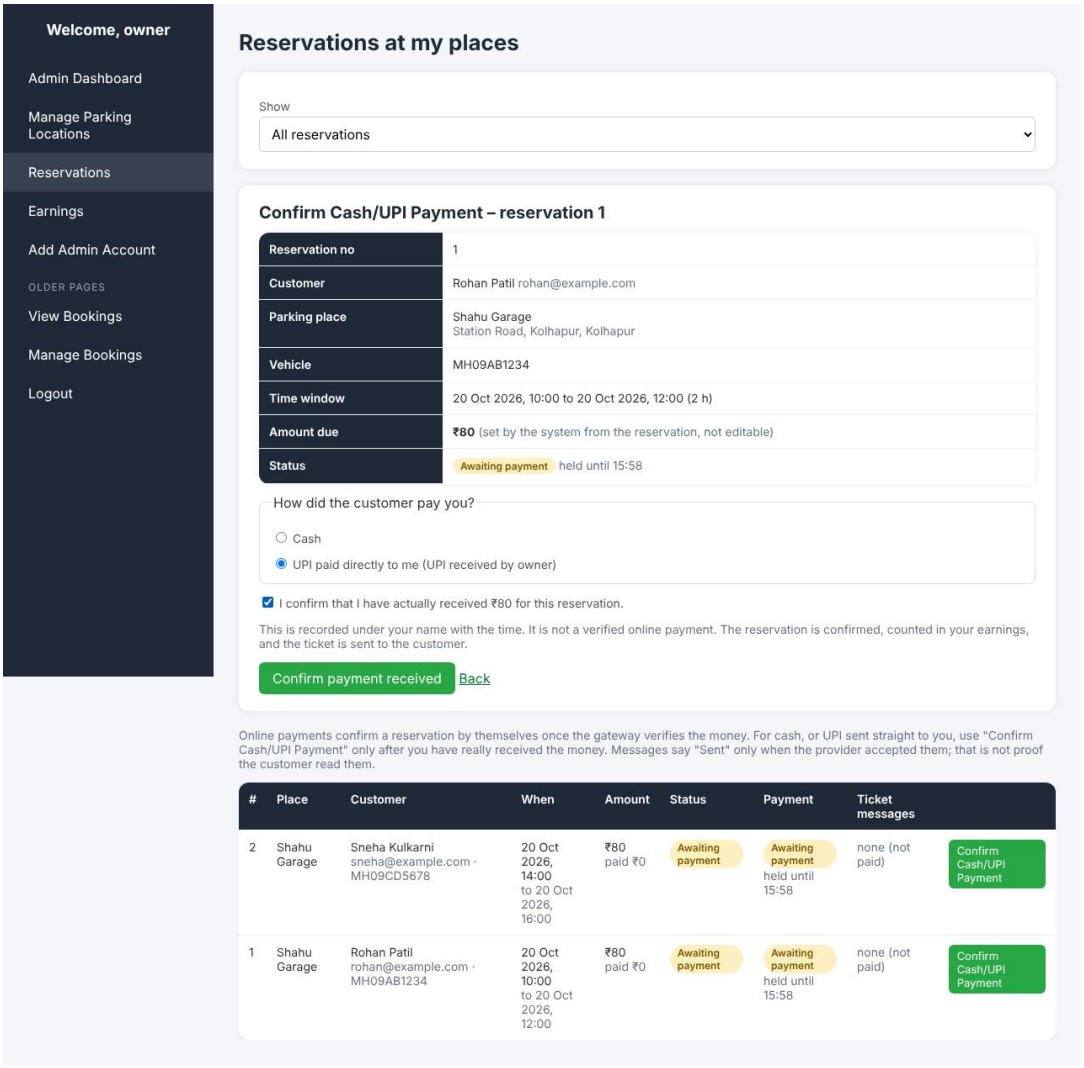


Fig. 8.12 Owner: Confirm Cash/UPI Payment review screen

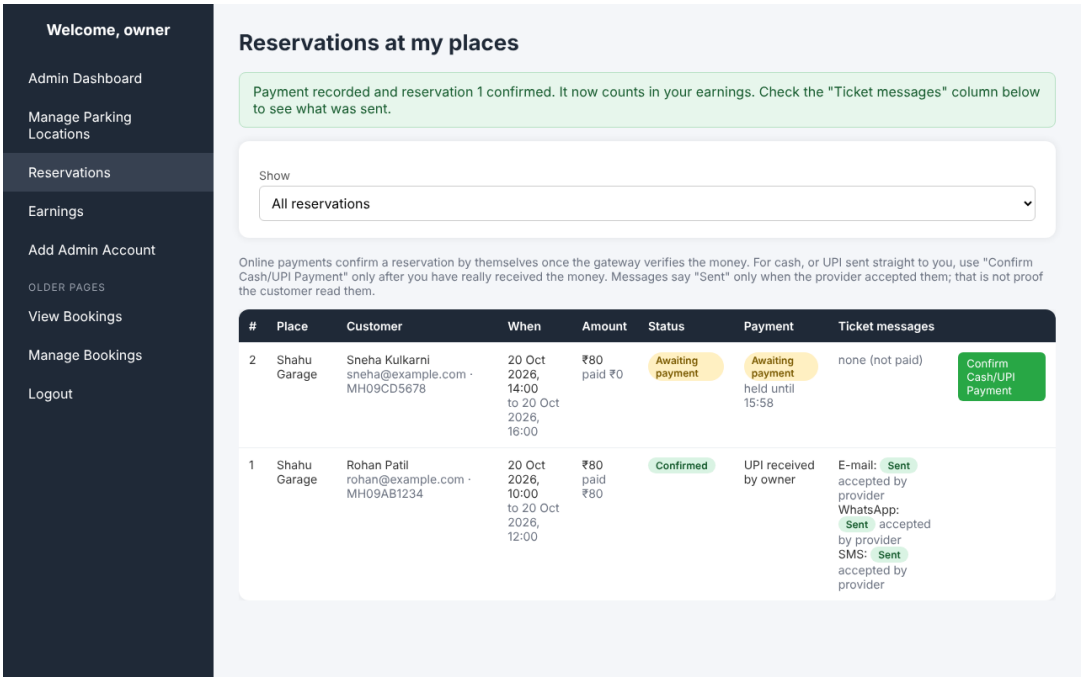


Fig. 8.13 Owner: after confirmation, with message status per channel

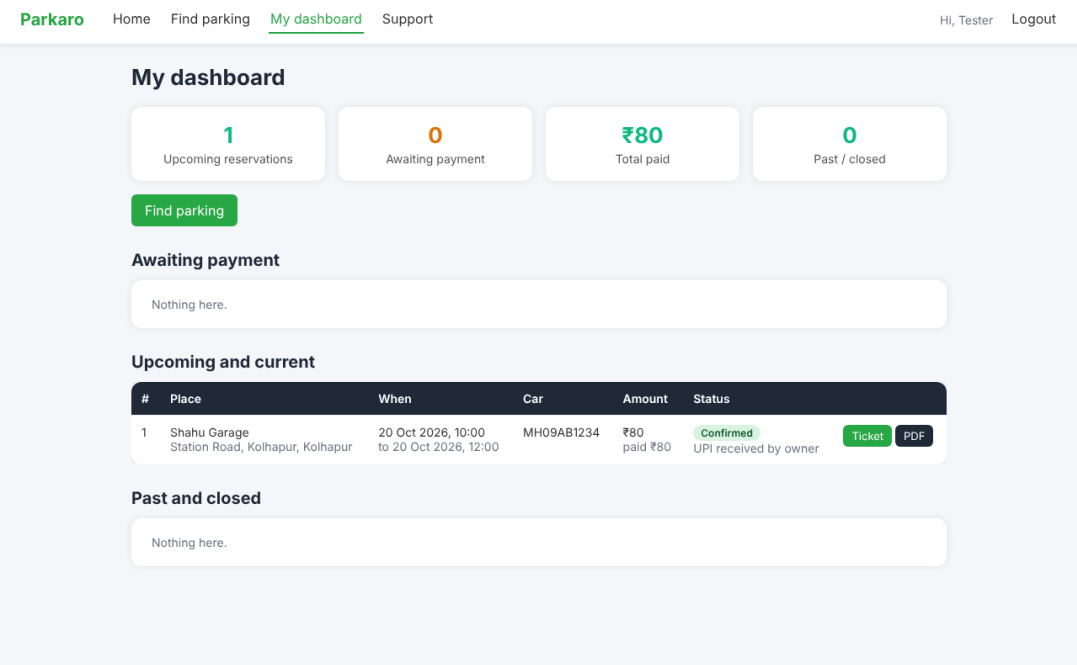


Fig. 8.14 User dashboard after confirmation: ticket, PDF and payment method

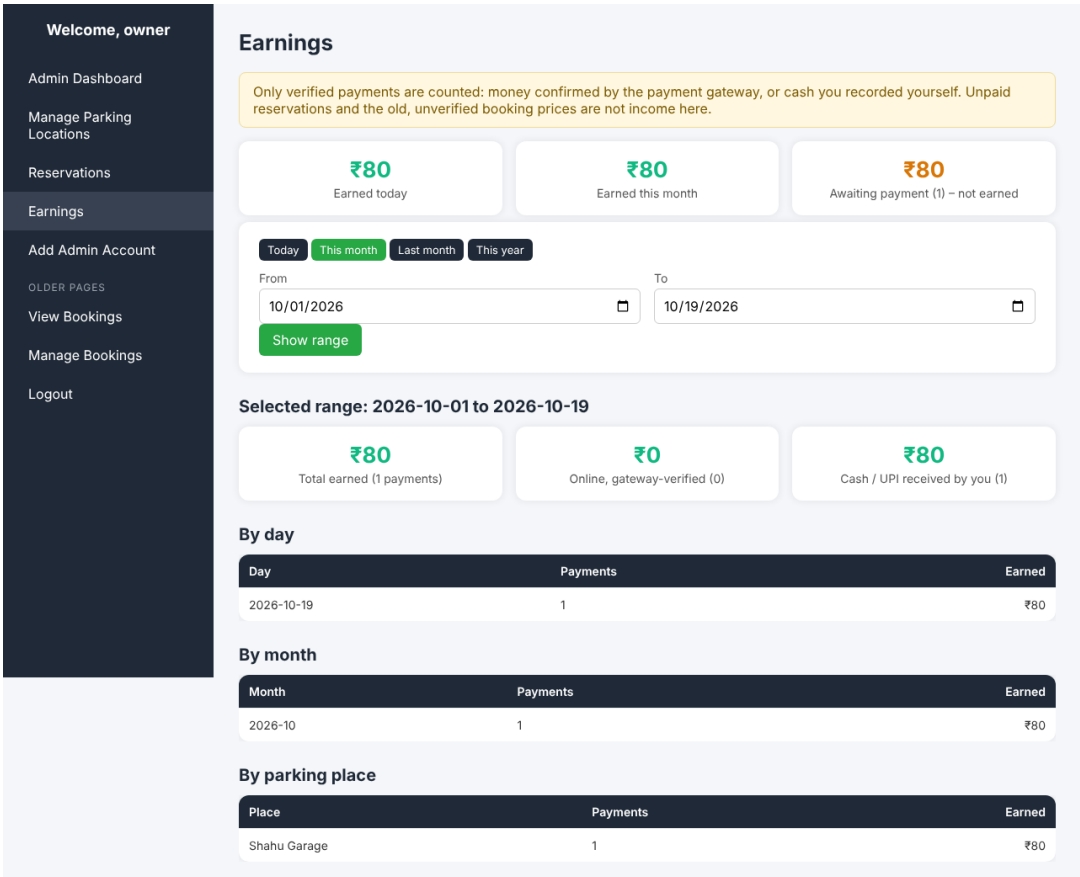


Fig. 8.15 Owner: earnings (verified payments only)

CHAPTER – 9: FEATURES

- **Find parking in three cities:** search by city and area, or use the device location; free public spots and reservable private places are clearly separated.
- **Live map and directions:** Leaflet and OpenStreetMap map with distance and a Google Maps directions link; no paid map API.
- **Owner portal:** owners register places with price, capacity, opening hours and accepted payment methods.
- **Server-controlled reservation:** time window, capacity and price are decided by the server; the space is held for 20 minutes.
- **Online payment with verification:** Razorpay (UPI, cards, net banking); never confirmed on the browser's word.
- **Pay at the place:** cash or UPI paid directly to the owner, confirmed by the owner on a review screen with a "money received" tick.
- **Ticket delivery:** e-mail, WhatsApp and SMS after confirmation, once per channel, with a secure ticket link and a PDF ticket.
- **Message status and retry:** owners see Sent / Failed / Not sent per channel and can retry safely.
- **Earnings report:** by day, month and place, counting only verified payments.
- **Security:** hashed passwords, prepared statements, CSRF tokens, signed ticket links, owner-scoped data.
- **Safe upgrade:** additive, repeatable database migrations; existing data is preserved.
- **Tested:** 938 automated checks covering the rules that matter for money and bookings.

CHAPTER – 10: LIMITATIONS

- **No live occupancy:** availability of reservable places is calculated from reservations, not from sensors or cameras. For free public spots, availability is unknown.
- **Free-spot data is static:** the 138 free spots are a built-in list; 68 of them are flagged in the project notes for on-site verification.
- **No refunds:** paid reservations cannot be cancelled and refunded from the system in this version. Money received for a reservation that could not be confirmed is listed for the owner to refund manually.
- **Owner confirmation relies on trust:** cash and UPI-to-owner payments are confirmed by the owner's own action; the system records who did it and when, but cannot check the money.
- **Message delivery is not proven:** "Sent" means the provider accepted the message; delivery to the phone is not confirmed. The phone number is not verified by an OTP.
- **Provider rules:** SMS to Indian numbers needs DLT registration, and WhatsApp business messages need an approved template; these are set-up tasks outside the code.
- **Internet dependency:** map tiles, payment checkout and messages need an internet connection.
- **Legacy pages remain:** some older pages from the diploma version (city pages, old dashboards) still exist and use the older database code.
- **Not load-tested:** the system has been tested for correctness, not for thousands of simultaneous users.

CHAPTER – 11: FUTURE SCOPE

- Refunds and cancellation of paid reservations through the Razorpay refund API.
- Live occupancy using IoT sensors, cameras or number-plate recognition (our AI & Data Science specialisation fits here), and parking-demand prediction from booking history.
- Phone-number verification by OTP, and a native Android/iOS app.
- More cities and an owner-side check-in / check-out screen with QR codes on the ticket.
- Payout reports and invoices for owners; ratings and reviews for places.
- Moving remaining legacy pages to the new modules and adding a MySQL job to run the automated tests on every change.

CHAPTER – 12: CONCLUSION

Parkaro started as a diploma project that displayed parking locations on a map. In this project we rebuilt it into a working system for finding, reserving and paying for parking in Kolhapur, Ajara and Gadhinglaj. Drivers can find free and private places, reserve a space for a time window and pay online or at the place; owners can list their places, decide how they are paid and see verified earnings; and customers receive their ticket by e-mail, WhatsApp and SMS.

The most important lesson of the project is to design around trust. The server, not the browser, decides price and availability; a reservation is confirmed only after a verified payment; repeating a request never repeats a payment or a message; and a failure of an outside service never damages a booking. These rules were turned into 938 automated checks, all of which pass. The parts that depend on real Razorpay, SMTP, Twilio and WhatsApp accounts still need to be tested with live credentials, and the setup guide and manual checklist in the appendices describe how.

The project gave us practical experience in full-stack web development, database transactions, payment integration, security and testing, and it leaves a solid base for adding sensors, AI-based prediction and mobile apps in the future.

CHAPTER – 13: REFERENCES

Research papers (carried over from the diploma report):

[1] M. M. Rashid, A. Musa, M. A. Rahman, and N. Farahana, "Automatic parking management system and parking fee collection based on number plate recognition," International Journal of Machine Learning and Computing, vol. 2, no. 2, pp. 93–98, Apr. 2012.

[2] A. M. Al-Turjman, H. Zahmatkesh, and L. Mostarda, "Smart parking in IoT-enabled cities: A survey," Sustainable Cities and Society, vol. 49, p. 101608, 2019.

[3] M. Ibrahim, Y. B. Zikria, M. Khalil, et al., "Design and development of a smart parking system using IoT and cloud technologies," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 11, no. 3, pp. 514–520, 2020.

[4] D. Kaur and E. Walia, "Smart parking system based on reservation and real-time monitoring," International Journal of Scientific and Research Publications, vol. 5, no. 12, pp. 1–5, Dec. 2015.

[5] A. Khanna and R. Anand, "IoT based smart parking system," in Proc. Int. Conf. Internet of Things and Applications (IOTA), Pune, India, Jan. 2016, pp. 266–270.

[6] H. Elmaghraby, A. Salama, and S. E. Hassan, "Smart parking guidance system using image processing," Alexandria Engineering Journal, vol. 59, no. 6, pp. 5099–5107, 2020.

[7] M. Kianpisheh, N. Y. Y. Idris, A. Saad, and P. Kong, "Smart parking system (SPS) architecture using wireless sensor network technology," International Journal of Software Engineering and Its Applications, vol. 6, no. 3, pp. 51–58, Jul. 2012.

Documentation and web references:

[8] Leaflet – an open-source JavaScript library for interactive maps. https://leafletjs.com

[9] OpenStreetMap. https://www.openstreetmap.org

[10] PHP Manual. https://www.php.net/manual

[11] MariaDB Knowledge Base. https://mariadb.com/kb

[12] Razorpay Documentation – Payment Gateway, Orders, Webhooks. https://razorpay.com/docs

[13] Twilio Documentation – SMS and WhatsApp. https://www.twilio.com/docs

[14] Google Maps URLs (directions links). https://developers.google.com/maps/documentation/urls

[15] OWASP Cheat Sheet Series (CSRF, SQL injection, password storage). https://cheatsheetseries.owasp.org

[16] How to build a real-time parking availability system. https://www.cprime.com/resources/blog/how-to-build-a-real-time-parking-availability-system/

[17] Parking spot finder app development. https://www.spaceotechnologies.com/blog/parking-spot-finder-app-development/

[Please check every reference against your college's citation format and add or remove entries; the web addresses should be opened once before submission.]

APPENDIX A: SETUP AND MIGRATION ON XAMPP

1. Copy the project folder to C:\xampp\htdocs\parkkaro_upgraded\project and start Apache and MySQL in the XAMPP Control Panel.
2. Create the database (for example parkaro) in phpMyAdmin and import database/schema.sql if it is a new installation.
3. Copy .env.example to .env and fill in the database name and an application key. Never share .env.
4. Back up the database (phpMyAdmin > Export).
5. Run php tools/check_migration.php to see what is applied.
6. Run php tools/apply_migration.php 001 --yes, then php tools/apply_migration.php 002 --yes. Both are additive and safe to repeat.
7. Run php tools/check_migration.php again; both migrations should be reported as fully applied.
8. Create the first admin with php tools/create_admin.php.
9. Open http://localhost/parkkaro_upgraded/project/home.html.

APPENDIX B: CONFIGURATION (.env) – NO REAL SECRETS HERE

Setting	Purpose
DB_HOST, DB_USER, DB_PASS, DB_NAME	Database connection
APP_KEY, APP_TIMEZONE	Signing key for tickets and sessions; time zone (Asia/Kolkata)
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Razorpay API keys (use TEST keys while developing); online payment is off if empty
APP_BASE_URL	Public address of the site, used in ticket links
MAIL_HOST, MAIL_PORT, MAIL_SECURE, MAIL_USER, MAIL_PASS, MAIL_FROM, MAIL_FROM_NAME	SMTP e-mail (for Gmail use an App password)
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_SMS_FROM, TWILIO_WHATSAPP_FROM, TWILIO_WHATSAPP_CONTENT_SID	SMS and WhatsApp through Twilio (WhatsApp sandbox for testing; approved template for real customers)

APPENDIX C: MANUAL TESTING CHECKLIST (REAL SERVICES)

1. Apply migrations 001 and 002 and confirm with check_migration.php.
2. As an owner, register a place with both payment methods; edit it and remove “At the place”; confirm that the Pay page of a user shows only online payment.
3. Put Razorpay TEST keys in .env, reserve as a user, pay with a test UPI/card, and confirm that the reservation becomes confirmed and the ticket appears in My dashboard.
4. Fill the MAIL_* settings and confirm that the e-mail arrives once; refresh the result page and confirm that no second e-mail arrives.
5. Add the Twilio sandbox settings, reserve with your mobile number and WhatsApp/SMS ticked, pay, and confirm both messages arrive.
6. Put a wrong MAIL_PASS, confirm a payment, check that the booking is still confirmed and the Ticket messages column shows Failed with a reason; fix .env and press Retry messages.
7. Reserve at an at-the-place location; as the owner use Confirm Cash/UPI Payment (UPI received by owner), tick “money received” and confirm; check the user dashboard and the messages.
8. Let a reservation’s 20-minute hold run out and confirm that the owner cannot confirm it and is told why.
9. Check Earnings: every verified payment counts once; pending reservations are not counted.
10. Run the automated tests: php tests/sqlite/run.php, pages.php, auth.php, tests/unit/*.php, tests/http/run.php.